MLPerf EDU

An Executable Benchmark Suite for ML Systems Education

Vijay Janapa Reddi
Harvard University

Artifact snapshot July 15, 2026

REVIEW DRAFT. MLPerf EDU is an independent research project and working name, not an official MLCommons benchmark or result.

Abstract

Benchmarking machine-learning systems requires holding task, data, metric, and quality targets constant, but production suites enforcing these properties demand computing resources unavailable to individual students or reviewers. The standard educational substitute is a collection of toy models, which fits local resource constraints but discards the quality gate, allowing a faster runtime claim to hide degraded accuracy or a skipped validation split. MLPerf EDU solves this by providing an executable specification and fail-closed admission rule that fits on a single laptop without compromising the evaluation contract. Every workload inherits its quality target from an upstream authority, and the suite enforces these bounds by treating any missed threshold or procedural violation as a failure rather than missing data. The resulting portfolio spans 14 workloads across 8 suites, yielding 12 evidence cases and 6 recorded misses, exposing the friction of real systems evaluation. By enforcing this strict Evaluation Tuple natively, the suite allows a student to construct, execute, and defend a valid performance claim in an afternoon, providing a practical foundation for Benchmarking Literacy. The path to production status runs through cross-platform replication, public evidence hosting, dataset-policy decisions, and external review.

1 Introduction

Machine-learning systems span training, serving, retrieval, graph analytics, time-series forecasting, and embedded inference. Evaluating these systems requires holding task semantics and quality constant while recording the model, data, hardware, scenario, and measurement procedure. While MLPerf established this discipline for production systems (Mattson et al., 2020; Reddi et al., 2019; Banbury et al., 2021), its submission infrastructure often exceeds the resources of a course or an individual artifact review. The standard educational substitute is a collection of small models, which fits local constraints but discards the quality gate. A student can then report a faster run after altering the data, relaxing accuracy, or timing a warmup pass. The missing learning objective is Benchmarking Literacy, the ability to construct, inspect, reproduce, and defend an evidence-backed performance claim.

MLPerf EDU addresses this gap with an executable specification. Its scope is locally executable, quality-gated evaluation for teaching and studying single-node ML systems.

Definition 1 (Evaluation Tuple)**.** *A benchmark configuration $\mathcal{S} = (W, D, M, H, S, C, P)$ binds the workload, data, model, hardware, scenario, constraints, and measurement protocol. It produces $\mathcal{M} = (Q, L, T, E)$, containing task quality, latency, throughput, and an energy observation when calibrated. An evidence package $\mathcal{E}$ records the configuration, measurements, source state, assets, and model lineage.*

The framework enforces this tuple through a Fail-Closed Admission Rule. A result r is admissible only when

$$\begin{aligned}\text{admit}(r) = G_{\text{sem}}(r) \wedge G_{\text{protocol}}(r) \\ \wedge G_{\text{provenance}}(r) \wedge G_{\text{state}}(r),\end{aligned}$$

where the semantic gate is task quality for score-bearing cases and functional correctness plus model lineage for performance-bearing cases. The remaining terms check the declared protocol, content-bound evidence, and stable host conditions. Fail-closed means the default is exclusion; a case that satisfies none of the gates by failing silently is treated as a failure rather than as missing data.

The paper makes six contributions.

1. **An executable benchmark contract.** The Evaluation Tuple and Fail-Closed Admission Rule form the basis of the runner, grading, host-state, and reporting invariants.
2. **A backward-designed workload portfolio.** The 14 workload identities cover language, retrieval, recommendation, reinforcement learning, graph learning, time series, vision, and embedded inference.
3. **Three execution intents.** The `min`, `max`, and `pro` profiles separate setup checks, comparable classroom runs, and research variants while preserving stable workload identities.
4. **Auditable and lineage-bound evidence.** Executions emit JSON, HTML, CSV, and provenance manifests that bind source, assets, hardware, metrics, and training checkpoints.
5. **A measured basis for single-run acceptance.** A determinism study over 200 executions shows inference quality is bit-identical across seeds and backends, justifying single-run acceptance, whereas training requires multiple trials.
6. **Measured laptop references.** The 12 retained evidence cases traverse the public command path and reproduce their inherited target on a single machine.

The goal is to make professional evaluation practice inspectable at a scale students and researchers can run locally. Figure 1 summarizes the system.

The paper follows this contract from gap to evidence. Section 2 positions the work against existing benchmark surfaces. Section 3 states the design principles that keep a laptop-scale workload faithful to its upstream task, and Section 4 describes the implementation. Section 5 defines the workload portfolio. Section 6 develops the three execution profiles. Section 7 reports the measured evidence, Section 8 the classroom progression, and Section 9 what the evidence does not yet establish.

2 Related Work

Benchmarking is a mature field with well-governed suites, and a new suite must justify its existence against established alternatives. Existing evaluation surfaces generally force a trade-off. Suites that enforce fixed task quality require compute budgets and submission infrastructure that exceed educational constraints. Conversely, suites that fit within a course's budget typically drop the quality gate, abandoning the core property that makes systems results comparable. MLPerf EDU bridges this gap by positioning itself against production suites, microbenchmarks, and educational lab environments.

Production suites. The official MLPerf Training and Inference suites established quality-constrained, scenario-aware performance evaluation at production scale (Mattson et al., 2020; Reddi et al., 2019). Their rules elevate quality targets, execution scenarios, and disclosure requirements from reporting conventions to requirements for result validity (MLCommons, 2026). MLPerf Tiny applies this methodology to constrained embedded environments (Banbury et al., 2021). MLPerf EDU inherits this rigorous discipline and selected workload contracts while targeting locally executable, single-node evaluation. It does not attempt to reproduce cluster-scale bottlenecks on a laptop. Instead, it asks what evaluation invariants survive when the compute budget shrinks and the reviewer is an instructor rather than a formal submission committee. Table 1 summarizes this positioning.

Microbenchmarks and leaderboards. DAWNBench introduced time-to-accuracy and cost-to-accuracy, connecting system performance to a fixed task-quality threshold (Coleman et al., 2019). MLPerf EDU retains that quality-first rule but targets fixed classroom exercises with inherited gates rather than leaderboards optimized across cloud providers. TorchBench provides broad PyTorch performance-regression coverage for framework development (Ansel et al., 2024). While MLPerf EDU shares its executable PyTorch surface, it adds task datasets, inherited quality gates, checkpoint lineage, and a governed portfolio. Where TorchBench asks whether a graph transformation regressed, MLPerf EDU asks whether a reported comparison is admissible under a declared task and quality contract. The two objectives remain complementary.

Educational lab setups. Framework-building courses make tensors, operators, and automatic differentiation inspectable. MLPerf EDU is complementary, treating the benchmark contract itself as the learning objective. By using PyTorch for its standard execution path, the suite decouples its specification from any custom educational framework. Students who build a framework from primitives still require a second discipline to know when a measured result is comparable to a baseline. MLPerf EDU provides that discipline without requiring the first.

Non-ML benchmarks. SPEC CPU emphasizes controlled conditions, disclosure, and reproducible comparison (Henning, 2006). TPC demonstrates that benchmark governance and reporting dimensions matter as much as the workload code (Poess and Floyd, 2000). MLPerf EDU applies these systems evaluation principles to machine learning, where task semantics and quality thresholds complicate the analysis. A classroom result that executes quickly but silently fails its accuracy target is no more ad-

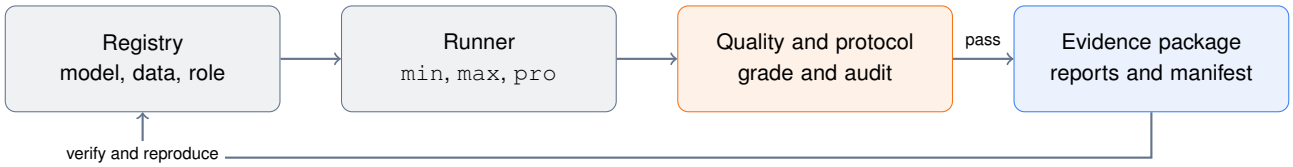

Figure 1. MLPerf EDU system architecture. Workloads flow through the unified CLI, asset, runner, grading, reporting, and provenance layers. The Evaluation Tuple binds each result to its model, data, hardware, scenario, constraints, and measurement protocol.

Table 1. Comparison with established benchmark surfaces. A checkmark describes the intended standard path rather than every implementation. The bottom row is the one no existing surface occupies: a complete single-laptop path that still holds an inherited quality gate.

	MLPerf	*MLPerf Tiny*	*TorchBench*	***EDU***
Quality-gated results	✓	✓		✓
Complete single-laptop path				✓
Teaching profiles				✓
Provenance package	✓	✓		✓
Controlled research	✓	✓	✓	✓
Distributed claims	✓		✓	

missible than a SPEC run compiled outside the reference flags.

The table exposes a gap rather than a crowded field. Suites that hold quality fixed run above the resource envelope of most courses and individual reviewers; suites that fit that envelope typically drop the quality gate rather than shrink the task. MLPerf EDU is designed to occupy the one row that keeps both properties at once.

That gap is what forced a build rather than an adoption. The missing artifact is not another collection of small models, which is abundant, but a suite in which the quality gate survives the reduction in scale. Nothing prevents a laptop-scale suite from inheriting an upstream target; what prevents it is that inheriting the target means accepting that some workloads will miss it, and a suite built to demonstrate its own workloads has an incentive to calibrate targets to what it can reach. The remainder of the paper describes a design that removes that incentive by construction, and reports what happens when the resulting contracts are run on one laptop, including where they fail.

3 Design Principles

Four principles guide the design of MLPerf EDU. Each addresses a specific failure mode that degrades a runnable script into an invalid benchmark. Together, these principles enforce the Evaluation Tuple and uphold the Fail-Closed Admission Rule.

3.1 Inherited Quality Contracts

Every quality target in the suite is inherited from an upstream authority. No target is ever set by this project. A suite that defines its own thresholds tends to define thresholds it can meet, adjusting tolerances to ensure local hardware passes. The result is a benchmark that cannot fail and therefore cannot inform.

Inheritance removes the discretion that makes drift possible. A target arrives from a peer-reviewed paper, a recognized leaderboard, an official evaluator, or a published model card. The target binds the dataset split, the exact metric, and the numerical threshold. The project's only decision is whether a candidate execution satisfies that threshold. When a laptop implementation falls short, the framework reports a miss. The framework cannot convert a failure into a pass because it lacks the authority to change the target.

This strict adherence enforces the Evaluation Tuple. In this snapshot, 6 contracts miss their inherited targets. Nudging a threshold or widening a tolerance would trivially rescue each one. Keeping them exposes the friction of real systems evaluation and gives students a suite in which failure is an inspectable outcome rather than a configuration error.

3.2 Transparent Benchmark State

Production benchmarks optimize for throughput using custom CUDA kernels, mixed-precision arithmetic, and aggressive operator fusion. These optimizations often render the underlying computation opaque. MLPerf EDU inverts this priority for its reference implementations, favoring pedagogical transparency over peak performance.

The standard execution path uses inspectable Python and PyTorch with explicit tensor operations and named dimensions. Thin adapters preserve the authoritative model and task contract rather than replacing them with approximations. Conformance is audited rather than assumed. For example, the current keyword-spotting

adapter preserves the DS-CNN topology and quality gate but is prediction-nonidentical to the pinned LiteRT graph, correctly blocking promotion. Students can still inspect its depthwise and pointwise convolutions as separate `nn.Conv2d` layers. The reference path establishes a readable baseline, not a performance ceiling. Students introduce optimized kernels or alternative runtimes through recorded `pro` configurations, which remain strictly subject to the Fail-Closed Admission Rule and the original semantic gate.

3.3 Provenance-Bound Evidence

A benchmark without real data teaches stress testing, not systems evaluation. Students must observe how data distributions affect convergence rates, final accuracy, and relative system ranking. Table 3 documents every dataset used by a canonical mode. The executable registry carries structured dossiers detailing pinned sources, dataset sizes, official splits, licensing terms, and digest policies.

The framework treats fetching and redistribution as separate decisions. A dataset can support a reproducible local run while its bytes remain excluded from a public evidence package, respecting distribution constraints while ensuring the Evaluation Tuple remains fully specified and verifiable.

3.4 End-to-End Execution Lineage

Isolated tools often provide only training or only inference, severing the causal link between a trained model and its deployment performance. MLPerf EDU preserves the complete pipeline whenever model lineage forms part of the workload contract.

Causal language-model training produces a quality-approved checkpoint. Subsequent full, prefill, and decode inference phases load that exact checkpoint, verifying its generated report, manifest, and digest prior to measurement. The registry centralizes metadata for the dataset, model, checkpoint, execution profile, and quality target. This design ensures deterministic, reproducible experiments that model the full lifecycle of an artifact. Training, full generation, prefill, and decode remain phases of a single workload identity. Treating them as separate identities would obscure their shared task semantics and checkpoint lineage, violating the integrity of the Evaluation Tuple.

4 Architecture and Implementation

The implementation follows a white-box design where a declarative registry selects assets, runners, scenarios, profiles, gates, evidence roles, and promotion rules. A unified command-line interface drives the fetch, audit, run, grade, verify, report, package, and validation operations. The registry also generates the flat `workloads.yaml` compatibility mirror, website tables, and paper snapshot. This single-source design ensures that disagreement between the manuscript and the executable contract results in a build failure rather than a documentation error.

```yaml
workload:
  id: causal-language-modeling
  suite: language
  profiles: [min, max, pro]
  runner: nanogpt
  dataset: tinyshakespeare
  quality_target:
    metric: cross_entropy_loss
    direction: lower
    threshold: 1.4697
    target_basis: literature
  public_status: experimental
```

Listing 1. Registry-driven workload metadata. Workload rows declare the user-facing selector, suite, profile coverage, assets, and quality policy.

4.1 Registry

The registry controls both models and data. Model records bind a user-facing identifier to an implementation or pinned upstream revision. Checkpoint-backed inference requires a lineage record containing the source training workload, task quality, and checkpoint digest, preventing a serving result from silently changing weights between repetitions.

Dataset recipes distinguish bundled, fetched, cached, synthetic, and restricted assets. Reports record the resolved mode and relevant digests. A `min` runner can use a reduced or synthetic input, but the public audit rejects that mode for score-bearing evidence. Dataset dossiers also record the citation, license, release status, and expected cache behavior.

4.2 Load Generator

The Python-native load generator implements inspectable scheduling for single-stream, offline, server, and multi-stream experiments (Reddi et al., 2019). It records warmup and measurement counts, latency samples and percentiles, throughput, and functional outcomes. Designed for teaching and local experimentation, it avoids opaque bindings where possible, serving as an inspectable substitute rather than a drop-in replacement for the official MLPerf LoadGen.

4.3 Power Measurement

Promotion campaigns record the power source, Low Power Mode, and sleep or wake state at each execution

boundary on supported hosts. A campaign refuses to start without external power and invalidates an affected attempt when the host state changes. These guards improve timing control but do not provide calibrated energy measurement. Absolute energy claims still require supported platform counters or an external meter with a disclosed calibration.

4.4 Merkle Provenance Engine

Each run produces a JSON report alongside human-readable HTML and CSV views. A `.provd.json` manifest binds the report and declared sidecars by size and SHA-256 digest. Hardware and software fingerprints, asset dossiers, quality metadata, and checkpoint lineage travel with the result. Packages rewrite machine-specific absolute paths so that verification works after extraction in a clean directory.

Hashing establishes integrity rather than authenticity. A self-consistent manifest can still be fabricated by its creator. Independent reruns, signed attestations, or controlled execution provide stronger trust. The framework states this boundary instead of claiming that checksums prevent fraud. The manifest provides unauthenticated integrity, not proof of origin or execution.

4.5 CLI

The CLI makes the Evaluation Tuple concrete. The suite, workload, variant, and profile select the model, data, scenario, constraints, and protocol. The commands below cover the normal review path.

```
$ mlperf doctor
$ mlperf list workloads
$ mlperf fetch \
    --workload causal-language-modeling \
    --mode training --profile max
$ mlperf run \
    --workload causal-language-modeling \
    --mode training --profile max
$ mlperf grade submissions/
$ mlperf verify submissions/run.provd.json
$ mlperf package submissions/run.provd.json
$ mlperf audit --policy public
```

Listing 2. CLI path from setup to verified evidence.

The same metadata drives individual runs and release presets. This reduces the chance that a paper-only configuration diverges from the executable path.

4.6 Validation Methodology

Validation is layered because a single green status can hide different kinds of failure. Unit and integration tests exercise runners, schemas, registry invariants, grading, provenance, packaging, and generators. Negative tests reject path traversal, changed source bytes, seed disagreement, incomplete functional gates, partial batch output, unavailable devices, unresolved dataset bytes, and timing evidence above its declared variation limit.

The `smoke` preset runs four representative `min` workloads. The `coverage` preset runs all 14 `min` workloads. The `max` profile selects canonical quality paths for the 9 promotion-scope workloads and bounded functional probes for the remaining 5. The `pro` profile selects the research workloads and repeats their `max` contracts under a separate execution target. Each phase verifies provenance and applies its declared quality or functional gate.

The full artifact test suite passes with two expected skips. The smoke and coverage presets passed 4/4 and 14/14 workloads. A retained one-measurement `pro` validation passed the four promotion-scope research workloads, their quality gates, provenance verification, and grading with zero warnings. The expanded `pro` plan adds the five bounded functional probes, whose individual `max` paths also pass. The 12 source-locked `max` cases all have retained evidence. A clean Python 3.12.13 wheel installed outside the checkout, loaded 14 workload definitions and 12 evidence records, executed the starter smoke path, and verified the resulting provenance.

The Quarto site renders 31 pages and passes 62 desktop and narrow-viewport checks. The paper generator binds this manuscript to the same evidence index and source lock. The strict public audit remains fail-closed because all 14 workloads are experimental. The validation supports internal consistency and reviewability, not external reproducibility. Hosted Linux CI, dataset-policy decisions, and independent reproduction remain separate gates.

5 Workload Suite

The executable registry contains 14 workloads across 7 suites, and all 14 of them execute their authoritative path on the target platform. No workload is environment-gated, so every contract in this section can be run and checked by a reader with the same hardware class. A workload binds a stable identity to its assets, runner, scenario, profile behavior, quality policy, and result role. Modes, phases, execution options, and research variations do not create new workload identities. This registry is the source of truth. The paper imports its counts and evidence table from a generated snapshot, which prevents prose from drifting away from executable metadata.

Each workload is run under one of three execution profiles, `min`, `max`, and `pro`, which separate installation checks from comparable runs and research variants.

Section 6 develops that separation and the reason it is a property of the run rather than of the workload.

The registry also separates public interpretation from execution success. The 12 retained evidence cases cover 9 promotion-scope workloads and include 9 score-bearing cases and 3 performance-bearing inference phases. The other 5 workloads have end-to-end functional evidence only. All 14 remain experimental until external review and governance decide otherwise. A successful execution can therefore anchor a lab or research comparison without being presented as an official benchmark score. This fail-closed distinction is important for small workloads because functional smoke data and reduced teaching exercises are not quality baselines.

Selection principles. Admission requires five properties. The task must have a recognizable upstream authority, fit the declared laptop envelope, preserve the system behavior it claims to expose, support a controlled pedagogical intervention, and provide an evaluator with a defensible gate. Resource accessibility alone is insufficient. A small model is rejected when its data, evaluator, or target must be invented for this project. Behavioral fidelity also does not imply production-scale equivalence. The framework makes no roofline or bottleneck classification without measurements, and a taxonomy field with no supporting measurement is left blank rather than estimated.

Alignment with MLCommons working groups. To ensure comprehensive coverage of modern machine-learning systems, the portfolio maps directly onto eight core MLCommons working group domains. Each suite exposes distinct systems characteristics and pedagogical learning objectives.

1. **Edge and Tiny (MLPerf Tiny).** Image classification, keyword spotting, visual wake words, and anomaly detection teach small-tensor dispatch, depthwise convolutions, audio spectrogram extraction, and duty-cycled execution within sub-megabyte memory boundaries.

2. **Language and LLM Serving (MLPerf LLM).** Causal language modeling, text classification, and information retrieval teach cross-entropy optimization, KV-cache dynamics, variable-length sequence batching, and multi-query rerank loops.

3. **AI Safety, Coding, and Agentic Systems (MLCommons Agents).** Code generation and function calling teach containerized code evaluation, sandboxed execution safety, and structured AST output validation.

4. **Graph Neural Networks (MLCommons Graph).** Graph node classification teaches irregular memory access patterns, sparse gather/scatter operations, and graph topology traversal on ogbn-arxiv.

5. **Time-Series and Scientific ML (MLCommons Science).** Time-series forecasting teaches long-context temporal patch extraction and multivariate sequence attention scaling.

6. **Recommendation Systems (MLPerf RecSys).** Recommendation teaches high-cardinality sparse embedding lookups, negative sampling, and dense interaction layers on MovieLens-20M.

7. **Generative AI and Diffusion (MLCommons Generative).** Image generation teaches iterative SDE/ODE diffusion sampler stepping and 50,000-image FID evaluation.

8. **Reinforcement Learning and Games (MLPerf RL).** Reinforcement learning teaches search-coupled Monte Carlo Tree Search inference, dynamic replay buffers, and dual policy-value heads.

The laptop envelope. Because admission depends on it, the envelope is stated rather than left implicit. A workload qualifies when it runs to completion on a single consumer machine with no accelerator beyond an integrated or laptop-class GPU, requires no manually licensed data, keeps its largest single asset small enough to fetch over an ordinary connection, and holds its working set inside the memory such a machine ships with. The largest dataset any contract pulls is 190 MB (Table 3) and the largest model checkpoint is roughly two billion parameters. That envelope is a constraint on admission, not a measured portability claim. Every timing number in this paper comes from one machine (Section 7), and the floor of the envelope, meaning the least capable machine on which the suite still completes, is untested and remains open work.

The rest of this section takes the workloads one at a time. Each paragraph answers the same four questions, because those are the questions that decide whether a reported number means anything. What is the benchmark? Why is it in the suite, given that resource fit alone is not a reason to admit a task? Where did its quality target come from? And which metric and evaluator decide pass or fail? Exact gate values and upstream authorities are listed in Table 2 and generated from the registry, so the prose below describes the provenance of each target rather than restating its digits.

5.1 Language and Retrieval

Causal language modeling. The task is character-level next-token prediction using the nanoGPT reference model. The contract maps to the MLPerf LLM

Table 2. Current workload contracts, generated from the executable registry. Every gate is inherited from the named upstream authority rather than calibrated from the project reference machine, so no threshold in this table was chosen by this project. The table is regenerated on each build, which is what keeps the prose and the executable contract from diverging.

Workload	Upstream Authority	Mode	Gate
`anomaly-detection`	MLCommons MLPerf Tiny, ToyADMOS, and DCASE	inference	ROC AUC $\geq$ 0.8500
`causal-language-modeling`	nanoGPT upstream repository	training; inference (full, prefill, decode)	loss $\leq$ 1.470
`code-generation`	Qwen and EvalPlus	inference	HumanEval+ pass@1 $\geq$ 0.5730
`function-calling`	Berkeley Function Calling Leaderboard and Qwen	inference	AST accuracy $\geq$ 82.92%
`graph-node-classification`	Open Graph Benchmark	training	test acc. $\geq$ 71.74%
`image-classification`	MLCommons MLPerf Tiny	inference	top-1 $\geq$ 85.00%
`image-generation`	NVIDIA EDM reference implementation	inference	FID $\leq$ 1.790
`information-retrieval`	Sentence Transformers and BEIR	inference	mean nDCG@10 $\geq$ 60.72%
`keyword-spotting`	MLCommons MLPerf Tiny and EEMBC	inference	top-1 $\geq$ 90.00%
`recommendation`	MLCommons MLPerf Training v0.5 recommendation	training	hit_rate_at_10 $\geq$ 0.6350
`reinforcement-learning`	Historical MLPerf Training MiniGo	training	move prediction $\geq$ 0.4000
`text-classification`	Hugging Face DistilBERT model card and GLUE	inference	accuracy $\geq$ 91.06%
`time-series-forecasting`	PatchTST authors and ETTDataset	training	test MSE $\leq$ 0.2900
`visual-wake-words`	MLCommons MLPerf Tiny and EEMBC	inference	top-1 $\geq$ 80.00%

Table 3. Dataset provenance and access. The table is generated from the registry dossiers. *Review* means the data can be fetched for a local run but is not bundled in a public package.

Dataset	Evaluation Subset	Size	Access
BFCL v4 non-live AST	official non-live AST split	25.0 MB	fetch; review
CIFAR-10	Tiny 200-example set	23.9 MB	fetch; review
ETTm1	official 12/4/4-month split	10.0 MB	fetch; review
HumanEval+	official 164-task set	1.0 MB	fetch
MiniGo self-play	self-play trajectories	generated	fetch; review
ToyCar (MLPerf Tiny)	Tiny 248-recording set	25.4 MB	fetch
Keyword spotting (MLPerf Tiny)	Tiny 1,000-example set	4.1 MB	fetch; review
Visual wake words (MLPerf Tiny)	Tiny 1,000-example set	2.7 MB	fetch; review
MovieLens 20M	leave-one-out, 999 negatives	190.0 MB	fetch-only
NanoBEIR	three English subsets	6.0 MB	fetch; review
ogbn-arxiv	official time split	83.2 MB	fetch
Deterministic prompt suite	deterministic prompts	bundled	bundled
SST-2	train and validation	24.5 MB	fetch; review
Tiny Shakespeare	90/10 train and validation	1.1 MB	fetch

working group domain. The systems learning objective teaches KV-cache dynamics and autoregressive sequence generation bottlenecks. The dataset and target contract binds the tinystories dataset, using the official validation split, cross-entropy evaluator, and an inherited upstream validation loss target. The empirical status is a recorded miss under the Fail-Closed Admission Rule because the measured score falls short of the inherited gate, exposing the friction of exact reproduction on constrained hardware.

Text classification. The task is binary sentiment classification using the DistilBERT reference model. The contract maps to the MLPerf LLM working group domain. The systems learning objective teaches fixed-sequence budget encoder inference and bidirectional attention patterns. The dataset and target contract binds the GLUE SST-2 dataset, its official validation split, accuracy evaluator, and the inherited upstream accuracy target from the model card. The empirical status is a pass, as the measured score meets the inherited gate, satisfying the Fail-Closed Admission Rule.

Information retrieval. The task is cross-encoder document reranking using the MiniLM-L6-v2 reference model. The contract maps to the MLPerf LLM working group domain. The systems learning objective teaches multi-query rerank loops and the cost structure of scoring many candidates per query. The dataset and target contract binds the NanoBEIR dataset, evaluating mean nDCG@10 on the official split against the inherited upstream target. The empirical status is a pass because the measured score exactly meets the inherited gate under the Fail-Closed Admission Rule.

Code generation. The task is function synthesis using the Qwen2.5-Coder reference model. The contract maps to the MLCommons Agents working group domain. The systems learning objective teaches containerized code evaluation and sandboxed execution safety. The dataset and target contract binds the HumanEval dataset, scoring pass@1 with the EvalPlus containerized evaluator against the inherited upstream target. The empirical status is a recorded miss under the Fail-Closed Admission Rule because the measured pass rate falls short of the inherited gate.

Function calling. The task is structured tool-call generation using the Qwen3 reference model. The contract maps to the MLCommons Agents working group domain. The systems learning objective teaches structured AST output validation and parsing agentic outputs. The dataset and target contract binds the BFCL AST dataset, using its non-live evaluation split and AST accuracy evaluator against the inherited upstream target. The empirical status is a recorded miss under the Fail-Closed Admission Rule because the measured score is below the inherited gate.

5.2 Graph and Time Series

Graph node classification. The task is node property prediction using a Graph Convolutional Network (GCN) reference model. The contract maps to the MLCommons Graph working group domain. The systems learning objective teaches irregular memory access patterns, sparse gather/scatter operations, and graph topology traversal. The dataset and target contract binds the ogbn-arxiv dataset, using the official time split and accuracy evaluator against the inherited upstream target. The empirical status is a pass because the measured score satisfies the inherited gate under the Fail-Closed Admission Rule.

Time-series forecasting. The task is multivariate long-horizon forecasting using the PatchTST reference model. The contract maps to the MLCommons Science working group domain. The systems learning objective teaches long-context temporal patch extraction and multivariate sequence attention scaling. The dataset and target contract binds the ETTm1 dataset, using the official test split and MSE evaluator against the inherited upstream target. The empirical status is a recorded miss under the Fail-Closed Admission Rule because the measured error exceeds the inherited gate.

5.3 Vision, Audio, and Embedded

Image classification. The task is object recognition using the ResNet8 reference model. The contract maps to the MLPerf Tiny working group domain. The systems learning objective teaches small-tensor dispatch and embedded-scale memory boundaries. The dataset and target contract binds the CIFAR-10 dataset, using the official 200-example accuracy set and top-1 accuracy evaluator against the inherited upstream target. The empirical status is a pass because the measured score meets the inherited gate under the Fail-Closed Admission Rule.

Keyword spotting. The task is spoken word recognition using the DS-CNN reference model. The contract maps to the MLPerf Tiny working group domain. The systems learning objective teaches audio spectrogram extraction and depthwise convolutions. The dataset and target contract binds the SpeechCommands dataset, using the official 1,000-example accuracy set and top-1 accuracy evaluator against the inherited upstream target. The empirical status is a pass, with the measured score meeting the inherited gate under the Fail-Closed Admission Rule.

Visual wake words. The task is always-on person detection using the MobileNetV2 reference model. The contract maps to the MLPerf Tiny working group domain. The systems learning objective teaches duty-cycled execution and sub-megabyte memory boundaries. The dataset and target contract binds the VWW dataset, using the official 1,000-example accuracy set and top-1 accuracy evaluator against the inherited upstream target. The empirical status is a pass because the measured score achieves the inherited gate under the Fail-Closed Admission Rule.

Anomaly detection. The task is unsupervised machine condition monitoring using an Autoencoder reference model. The contract maps to the MLPerf Tiny working group domain. The systems learning objective teaches small-tensor dispatch and feature reconstruction evaluation. The dataset and target contract binds the ToyADMOS dataset, using the official log-mel feature recipe, the ROC AUC evaluator, and the inherited upstream target. The empirical status is a pass because the measured score meets the inherited gate under the Fail-Closed Admission Rule.

Image generation. The task is unconditional image synthesis using the EDM reference model. The contract maps to the MLCommons Generative working group domain. The systems learning objective teaches iterative SDE/ODE diffusion sampler stepping and large-scale generated evaluation. The dataset and target contract binds the CIFAR-10 dataset, using the FID evaluator over 50,000 images against the inherited upstream target. The empirical status is a recorded miss under the Fail-Closed Admission Rule because the measured FID exceeds the inherited gate.

5.4 Recommendation and Reinforcement Learning

Recommendation. The task is item recommendation using the Neural Collaborative Filtering (NCF) reference model. The contract maps to the MLPerf RecSys working group domain. The systems learning objective teaches high-cardinality sparse embedding lookups, negative sampling, and dense interaction layers. The dataset and target contract binds the MovieLens-20M dataset, using the leave-one-out HR@10 evaluator against the inherited upstream target. The empirical status is a

Table 4. Execution profiles and their intended interpretation. The profile describes why a run was performed, not how hard it is. Workload identity, model, data, evaluator, and gate are identical across all three, so moving between profiles changes what may be claimed from a result rather than what was measured.

Profile	Interpretation
`min`	Fast correctness and setup path. No public score.
`max`	Comparable reference path with the declared assets and gates.
`pro`	Research variants and ablations with configuration-dependent comparability.

recorded miss under the Fail-Closed Admission Rule because the measured score falls short of the inherited gate.

Reinforcement learning. The task is board game self-play using the MiniGo reference model. The contract maps to the MLPerf RL working group domain. The systems learning objective teaches search-coupled Monte Carlo Tree Search (MCTS) inference, dynamic replay buffers, and dual policy-value heads. The dataset and target contract binds the Self-Play generated dataset, using the professional-move prediction evaluator against the inherited upstream target. The empirical status is a recorded miss under the Fail-Closed Admission Rule because the measured score does not meet the inherited gate under the reduced execution budget.

6 Execution Profiles

A benchmark suite that serves both a first-week installation check and a term-long architecture project has to distinguish those runs without letting either redefine the task. MLPerf EDU does that with three execution profiles. They describe execution *intent*, not difficulty, and they are a property of the run rather than of the workload, so the same workload identity, model, data, evaluator, and gate persist across all three.

`min` is a quick installation and plumbing check. It may use reduced or synthetic inputs and never supports a public score. `max` is the standard comparable path with the declared model, real data, gate, and protocol for promotion-scope workloads; for a functional-stage addition it remains a bounded setup probe that cannot support a quality or performance claim. `pro` is the research envelope for ablations, backend studies, repetitions, and user-controlled variants, and a `pro` result is comparable only when its recorded configuration defines the comparison.

The reason for three profiles rather than one is that the alternative encodes optimization into workload names. A suite without this separation accumulates identities such as `resnet-batched` or `gpt-compiled`, at which point the workload no longer denotes a task and cross-result comparison becomes a question about naming conventions. Keeping batching, compilation, precision, scheduling, and caching inside the profile leaves the workload identity stable and forces every optimization to be disclosed as configuration.

The separation also maps onto how the suite is actually used. `min` supports installation, pre-lab checks, and continuous integration. `max` is the classroom reference contract whose assets, evaluator, and gate must not change, which is what makes two students' submissions comparable. `pro` exposes repetitions, backends, compilers, and ablations for projects or architecture studies, where the configuration is the experiment and comparability is scoped to whatever the record discloses.

7 Experimental Results

The evaluation tests three claims implied by the design. First, an unchanged upstream task can meet its quality gate through the laptop implementation. Second, the score-bearing suite fits a practical local execution budget. Third, the framework distinguishes an executable result from one with enough repeated evidence to support a timing claim.

The evidence campaign evaluates 12 canonical cases on an Apple M5 Max host from source commit `163d42ee3d`. CPU and Apple MPS are used according to the declared case contract. Every retained run passed execution, grading, provenance, source, and semantic checks. Where a case was measured more than once, its timing spread is reported alongside the median; a single measurement is reported as one measurement and carries no repeatability claim.

A single accepted run decides a quality verdict, while any timing claim rests on repeated measurement. That asymmetry only holds if the quality metric is genuinely deterministic. We measured whether it is. Each of 6 inference workloads was run under 5 seeds on the default MPS backend and then repeated once on CPU, giving 36 executions in total. The design is deliberately not a full crossing of seeds and backends; it tests seed sensitivity at depth on one backend and then checks whether the second backend agrees at all. The quality metric did not move under either test. The largest spread across seeds was 0.0e+00 and the largest difference between backends was 0.0e+00. Every value was bit-identical.

Training does not behave this way. Sweeping 3 seeds across the 2 training workloads cheap enough to repeat, the seed spread exceeded the margin to the effective threshold in both, and in both the verdict changed.

Graph node classification is recorded as a pass at 0.7210 and falls to 0.7115 under another seed, below its tolerance floor. Time-series forecasting is recorded as a miss at 0.2924 and reaches 0.2889 under a third seed, inside its target. The verdict moves in both directions, so a single accepted run does not settle a training comparison.

We report this rather than averaging it away, and the suite is arranged so a student meets it. A production submission absorbs this variance into a many-run protocol, which is correct for publishing a number and unhelpful for learning why the protocol exists. Here the gap between a workload's seed spread and its distance to the gate is visible in the artifact, and reproducing it costs one command with a changed seed. A student who watches a passing workload miss, and a missing workload pass, has learned something about benchmark claims that no amount of prose about statistical rigor conveys. That is the intended use, and Section 8 builds an exercise on it.

The consequence for reading this paper is unchanged. A training row is one draw from a distribution about as wide as its distance to the gate, and should be read that way. Raising the accepted-run count for training contracts would narrow the reported interval, but it would also remove the most direct demonstration the suite offers of why one run is not evidence.

This matters most for the cases sitting closest to their contract line. Text classification exceeds its target by roughly two parts in a hundred million and information retrieval matches its target to eight decimal places, margins thin enough to look like luck. They are not. Those workloads evaluate a pinned checkpoint over a fixed set and consume no randomness, so they reproduce the published value rather than approach it. The single-run acceptance rule is therefore sound for inference, and the result also shows that quality, unlike timing, does not depend on the execution backend.

The campaign uses a create-once policy. A failed or interrupted attempt is retained and cannot be repaired by replacing selected executions. Power source, Low Power Mode, and sleep or wake changes are checked before and after each promotion execution. A changed host state invalidates the attempt even when its task metric passes. This separation is important because accuracy does not excuse unstable timing or uncontrolled measurement conditions.

7.1 Committed Reference Evidence

Figure 2 plots every one of the 14 contracts the suite executed against its own published target. Of those, 8 reproduce it and 6 fall short. Five of the six shortfalls are narrow, the widest under six percent relative, which is

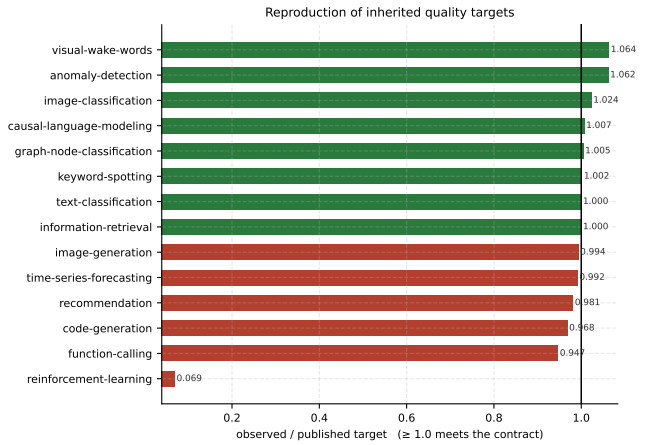

Figure 2. Reproduction of inherited quality targets. Each workload's observed value is divided by its own published target, so accuracy, ROC AUC, nDCG@10, hit rate, and MSE share one axis and a ratio of 1.0 is the contract line in every case. Direction is handled per metric, so a lower-is-better target such as test MSE is inverted before plotting. Most workloads sit within a few percent of the value published upstream, which is the expected result when a contract is inherited rather than retuned, and the bars below the line are recorded misses under the Fail-Closed Admission Rule rather than relaxed targets. Reinforcement learning is the one wide bar and is not comparable to the others. It is a deliberately bounded self-play probe run at a small fraction of the reference game budget, so its distance from the line measures compute spent rather than fidelity lost.

the expected spread when a contract is inherited rather than retuned.

Reinforcement learning is the exception and is a different kind of miss. Its adapter runs the pinned upstream reference, but the laptop configuration generates 16 self-play games per generation for four generations against a reference budget of 2,000 games per generation and up to 10,000 generations. The resulting value is far below the gate because the run is a bounded probe, not because the implementation diverges from the contract. It is plotted rather than hidden, since excluding an inconvenient result is precisely the discretion this framework is built to remove, but it should be read as a statement about budget and not about reproduction fidelity.

Of the executed contracts, 9 are admitted to score-bearing review, where 8 meet their inherited gates in every retained execution. The generator grades each retained result against the live registry contract, so a record graded under a superseded gate cannot publish its own looser bound; time-series forecasting is reported as a miss for exactly this reason. The score-bearing reference medians total 62.0 minutes for one score-bearing suite pass. The individual cases span 0.280 seconds to

Table 5. Committed project reference measurements. *Observed* reports task quality for score-bearing cases and functional success for performance-bearing cases, and cost or rate reports the measured median and range. Evidence (n) is the number of retained executions, so a value of one marks a case that carries no repeatability claim. None is an MLCommons-verified result.

Case	Role	Evidence (n)	Semantic Gate	Observed	Measured Cost or Rate	Timing CV	Device
Anomaly detection (inference)	Score	5	ROC AUC $\geq$ 0.8500	0.9029	inference s 0.2798 [0.2790, 0.2993]	3.08%	mps
Causal LM (decode)	Perf.	1	functional pass	pass	output tok/s 767.3	not established	cpu
Causal LM (full)	Perf.	1	functional pass	pass	output tok/s 695.5	not established	cpu
Causal LM (prefill)	Perf.	1	functional pass	pass	prefill tok/s 24.6k	not established	cpu
Causal LM (training)	Score	2	loss $\leq$ 1.470	1.459	train + eval s 2107.4 [2030.1, 2184.8]	5.19% diagnostic	mps
Graph classification (training)	Score	1	test acc. $\geq$ 71.74%	72.10%	train + eval s 719.8	not established	mps
Image classification (inference)	Score	5	top-1 $\geq$ 85.00%	87.00%	inference + eval s 7.837 [7.719, 7.921]	0.95%	cpu
Retrieval (inference)	Score	5	mean nDCG@10 $\geq$ 60.72%	60.72%	inference + eval s 17.97 [17.59, 18.32]	1.76%	mps
KWS (inference)	Score	5	top-1 $\geq$ 90.00%	90.20%	inference s 8.009 [7.970, 8.030]	0.31%	mps
Text classification (inference)	Score	5	accuracy $\geq$ 91.06%	91.06%	inference s 4.352 [4.269, 4.375]	1.10%	mps
Time-series forecasting (training)	Score	1	test MSE $\leq$ 0.2900	0.2924 **(miss)**	train + eval s 854.0	not established	mps
Visual wake words (inference)	Score	5	top-1 $\geq$ 80.00%	85.10%	inference s 0.7168 [0.7036, 0.7298]	1.41%	mps

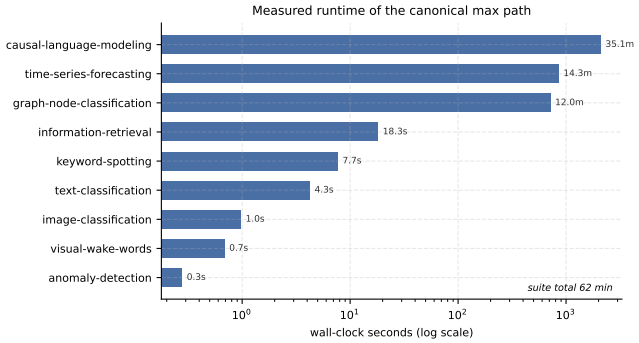

Figure 3. Measured runtime of the canonical max path. Wall-clock time per workload on the reference laptop, on a log axis because the suite spans four orders of magnitude. The fixed-checkpoint inference workloads complete in seconds, while the workloads that train from scratch take minutes. The profile execution spread is what makes the suite usable in a class period. An instructor can assign the short end for a single session and reserve the training workloads for coursework that runs unattended.

35.1 minutes (Figure 3), which permits short classroom exercises

without pretending that every training workload is instantaneous. Image classification, keyword spotting, machine-sound anomaly detection, visual wake words, text classification, and information retrieval were each measured over repeated fresh processes, with timing coefficients of variation from 0.31% to 3.08%. Time-series forecasting reproduces the published PatchTST recipe to 0.2924 test MSE against an unchanged 0.2900 reproduction point, so it is retained as a miss; no tolerance was introduced after the fact to close a 0.0024 gap. The two retained causal-training measurements have a 5.19% timing coefficient of variation, narrowly above the unchanged 5% promotion limit.

Neither training shortfall is a budget artifact, which matters because the obvious objection to a laptop-scale suite is that its misses are really just truncated runs. The per-epoch curves in Figure 4 rule that reading out. Both workloads reach their best value and then move away from it, so the shortfall survives a longer run rather than closing with one. The recommendation contract makes the point concrete. An earlier configuration read a still-rising curve as an epoch limit, and extending it to twenty epochs cost a further sixty-three minutes and returned a worse score.

Execution integrity is not adapter equivalence. The keyword-spotting record is valid measured evidence, but its committed three-resolver audit fails exact prediction parity. The quality-preserving but nonidentical adapter remains educationally useful while blocking promotion. The 90% quality gate is not a disagreement tolerance.

These results support the three evaluation claims stated at the start of this section, on a single machine. The inherited tasks remain reachable, the score-bearing reference path fits roughly one hour, and the Fail-Closed

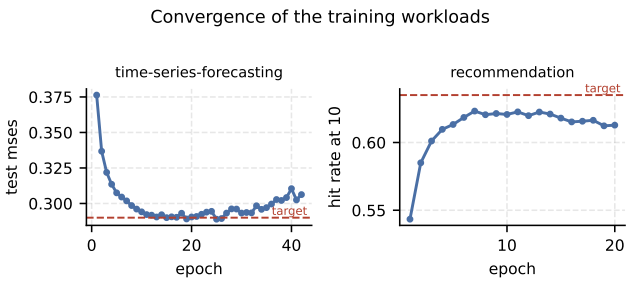

Figure 4. Convergence of the training workloads. Per-epoch task quality against the inherited target. Both panels show the same shape and it is not the one a longer run would fix. Time-series forecasting reaches the target line near epoch fifteen and then drifts back above it, and recommendation peaks at 0.6232 on epoch seven and declines thereafter. An earlier seven-epoch recommendation result read its still-rising curve as an epoch limit; extending the run to twenty epochs cost a further sixty-three minutes and returned a worse score, which is why the contract now caps the budget at the peak. Neither shortfall is a budget limit and neither was closed by relaxing a target. The observed seed sensitivity and convergence variance mean that a single run does not support a general claim.

Admission Rule keeps every case that missed its inherited gate out of score-bearing review rather than promoting it alongside the cases that passed. They do not establish performance portability. Task metrics are intended to transfer within declared numerical tolerances; runtime is not. The table therefore identifies each device, while retained reports record the complete hardware and software fingerprint. A second machine should be expected to reproduce the gate and protocol, not the Apple M5 Max throughput.

7.2 Inference Latency

Inference evidence is eligible only after correctness and model identity are established. Causal full, prefill, and decode inference load one retained quality-passing training checkpoint and record its SHA-256 digest. The evidence importer verifies the checkpoint package, source report, source manifest, and package digest before accepting any phase. This closes a common loophole in which inference silently uses random, stale, or separately trained weights.

Full inference measures complete autoregressive requests. Prefill measures fresh KV-cache construction for a fixed 256-token prompt. Decode measures cached-token generation after a fixed prompt. Reports retain median, p90, and p99 latency together with phase-appropriate token throughput and sample counts. The three retained CPU measurements report 695.5 output tokens/s for full inference, 24.6 thousand prefill tokens/s, and 767.3 output tokens/s for decode.

Each phase passes its functional and lineage gate, but each currently has one retained execution. The rates are therefore provisional system observations, not repeatability evidence or portable performance targets. Their value lies in their checkpoint, protocol, sample, and hardware context.

8 Classroom Use

The benchmark contract is itself the teaching object. The intended progression has four stages. Students **reproduce** a fixed profile and verify its report and provenance manifest, **diagnose** where time is spent across data loading, model execution, and request scheduling, **optimize** one systems variable while preserving the declared quality gate, and then **defend** the resulting comparison against the evidence supporting it. This structured approach develops Benchmarking Literacy. The profiles map onto that progression directly. An instructor can use `min` as a pre-lab environment check, `max` as the fixed reference for a bounded assignment, and `pro` as the disclosure envelope for an open-ended project, while the workload identity and semantic gate stay fixed throughout. Optimization therefore changes the system under test, never the definition of success.

The **defend** stage has a concrete instrument, and it comes from the suite's own measured behavior. Two training workloads have a seed spread wider than their distance to the gate, so a student who reruns graph node classification under a different seed watches a passing result miss, and a student who reruns time-series forecasting watches a recorded miss pass. Both take one command and a changed seed. The exercise asks a question the reader of any benchmark table should ask. Given this spread, which comparisons does this evidence actually support? A student who has seen a verdict move learns why a single number is not a claim, which is difficult to teach from a suite where every result is stable.

Three standalone labs make that progression concrete. One profiles a deliberately inefficient training loop and requires the submission to carry both configurations, task-quality outcomes, timing distributions, and an explicit statement of comparability. One implements a small inference system-under-test interface and compares naive autoregressive decoding against KV-cache decoding, passing only when both paths emit identical tokens for every measured query. One contrasts a dense model with a sparse mixture-of-experts model at similar headline parameter counts, reporting active parameter fraction alongside throughput to show why parameter count alone is not a systems cost model. Each lab has a deterministic offline smoke mode that exercises real for-

ward, backward, inference, or parity paths. Those smoke paths establish software regression coverage rather than classroom efficacy. This draft claims no measured learning outcomes; course pilots, accessibility review, and instructor studies remain future work.

9 Limitations and Future Work

The framework admits only contracts that fit local hardware without changing their defining task. That choice preserves authority and access, but it does not capture every production bottleneck. Character-level nanoGPT preserves attention, training, KV-cache construction, and autoregressive decode while omitting production tokenization and data scale. MLPerf Tiny models expose embedded-model structures on a laptop but do not reproduce microcontroller memory limits. Distributed and datacenter-scale claims are outside v0.1.

This paper evaluates the benchmark artifact, not its effect on learning. Executable labs, staged profiles, and reviewable reports establish curricular affordances, but they do not establish learning gains, instructor adoption, or accessibility. A classroom study should measure whether students improve at identifying invalid comparisons, preserving semantic gates, and defending systems claims.

Timing is reported from the measurements each case actually has, and a case measured once is reported as measured once. Promotion to a published baseline would still require repeated timing under stable host conditions, asset policy, and portable evidence, which v0.1 does not claim.

The current references come from one Apple M5 Max MacBook Pro with 64 GB of memory and 18 physical cores. Runtime is platform-specific, and every timing and repeatability number here characterizes that machine rather than the suite. Quality is better established. The determinism study shows the metric unchanged across seeds and across the MPS and CPU backends, so numerical drift between backends is measured rather than assumed for the inference cases. That leaves timing, training-workload seed sensitivity, and any drift specific to a different vendor's arithmetic as open questions. Cross-platform work should include Linux CPU and CUDA systems, clean-wheel installation, and repeated timing on each.

The paper argues that production suites exceed a course budget without measuring the alternative. A direct comparison, attempting an established production benchmark at reduced scale on the same laptop and reporting where it succeeds or fails, would test that premise rather than assert it. This is out of scope for v0.1 because the comparison is only meaningful against a submission-grade configuration of the other suite, which carries its own review and disclosure obligations. Until it is done, the accessibility argument rests on the operational requirements those suites document rather than on a measurement made here.

Several datasets remain fetch-only or require a release-policy decision. The project also needs an authoritative license for the complete published artifact, public hosting for larger evidence packages, and external decisions about the name and any relationship to MLCommons. Technical completion does not settle those governance questions.

Power-state guards improve the internal validity of timing campaigns but do not provide calibrated energy. Defensible joule claims require supported platform counters or an external meter. The five functional-stage additions now exercise recommendation, reinforcement learning, code generation, image generation, and function calling end to end, but they provide no authoritative quality or timing claim. Each must clear its unchanged quality contract before entering the promotion and repeatability protocol described in Section 7.

9.1 Threats to Validity

Internal validity. Where a case was measured over repeated fresh processes, that repetition reduces sensitivity to a single timing sample; where only one measurement exists, no repeatability claim is made and the report says so. Shared implementation error can remain across every case regardless of how many times it was run. Provenance digests detect byte changes and path drift; they do not prove that the computation occurred.

External validity. One laptop establishes local feasibility, not performance portability. The evidence records hardware and software fingerprints so later systems can test both quality transfer and timing variation.

Construct validity. The framework assumes that practicing controlled comparisons on reduced workloads improves production benchmarking judgment. That claim is plausible but unmeasured. The selection process may also favor tasks with compact pretrained checkpoints or established small evaluation sets. Formal course studies, independent workload review, and external artifact evaluation are needed to assess both effects.

10 Conclusion

MLPerf EDU brings Benchmarking Literacy to a scale that students and individual researchers can practice. Its central contribution is not a list of small models. It is an executable Evaluation Tuple that binds model, data, hardware, scenario, quality, protocol, and evidence before a performance claim is accepted. The suite applies a

Fail-Closed Admission Rule, taking every quality target from an upstream authority so the project retains no discretion to move a threshold it fails to reach.

The present implementation contains 14 workloads across 7 suites, of which 9 promotion-scope workloads carry 12 canonical evidence cases, while 5 additions have functional evidence only. In all, 8 of 9 score-bearing cases pass their quality gates, 1 is retained as a disclosed miss, and all performance-bearing phases pass their functional and lineage gates. Repeated timing, where collected, varies between 0.31% and 3.08%. Measuring the suite also produced a result about benchmark protocol rather than about any workload in it. Inference quality is bit-identical across seeds and across two execution backends, so a single run settles it, while both training workloads swept have a seed spread wider than their margin to the gate and flip verdict under a different seed. A protocol that accepts one run is therefore right for part of this suite and wrong for the rest, which is a distinction worth carrying into any small-scale benchmark. Those results make the benchmark ready for external technical review, not for a claim of official status. Public evidence hosting, cross-platform reproduction, dataset policy, licensing, and MLCommons feedback remain visible next steps.

The intended educational value lies in the workflow students practice. They reproduce a baseline, diagnose a bottleneck, change one variable, preserve quality, and defend comparability with artifacts. That workflow can outlast any particular model or hardware generation. Demonstrating its learning effect is the next empirical question.

Reproducibility

The repository contains the executable registry, reference runners, evidence summaries, website, labs, tutorial, and paper generator. The public entry point is `mlperf`. Runs produce JSON, HTML, CSV, and `.provd.json` artifacts. The paper imports generated registry counts and reference values so its quantitative claims remain synchronized with the committed class-labeled evidence. The larger raw packages are retained for local reviewer handoff and do not yet have public URLs.

References

Jason Ansel, Edward Yang, Horace He, Natalia Gimelshein, Animesh Jain, Michael Voznesensky, Bin Bao, Peter Bell, David Berard, Evgeni Burber, et al. TorchBench: Benchmarking PyTorch with High API Surface Coverage. *arXiv preprint arXiv:2401.16422*, 2024.

Colby Banbury, Vijay Janapa Reddi, Peter Torelli, Jeremy Holleman, Nat Jeffries, Csaba Kirber, et al. MLPerf Tiny Benchmark. In *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks (NeurIPS)*. Curran Associates Inc., 2021.

Cody Coleman, Deepak Narayanan, Daniel Kang, Tian Zhao, Jian Zhang, Luigi Nardi, Peter Bailis, Kunle Olukotun, Christopher Ré, and Matei Zaharia. Analysis of dawnbench, a time-to-accuracy machine learning performance benchmark. *ACM SIGOPS Operating Systems Review*, 53(1):14–25, 2019. ISSN 0163-5980. doi: 10.1145/3352020.3352024. URL https://doi.org/10.1145/3352020.3352024.

John L Henning. Spec cpu2006 benchmark descriptions. *ACM SIGARCH Computer Architecture News*, 34(4):1–17, 2006. ISSN 0163-5964. doi: 10.1145/1186736.1186737. URL https://doi.org/10.1145/1186736.1186737.

Peter Mattson, Christine Cheng, Gregory Diamos, Cody Coleman, Paulius Micikevicius, David Patterson, et al. Mlperf: An industry standard benchmark suite for machine learning performance. *IEEE Micro*, 40(2):8–16, 2020. ISSN 0272-1732, 1937-4143. doi: 10.1109/mm.2020.2974843. URL https://doi.org/10.1109/mm.2020.2974843.

MLCommons. MLPerf Inference Rules, 2026. URL https://github.com/mlcommons/inference_policies/blob/c547732b539cb3a14cc5680597714c8c1df4cad0/inference_rules.adoc. Accessed July 13, 2026.

Meikel Poess and Chris Floyd. New tpc benchmarks for decision support and web commerce. *ACM SIGMOD Record*, 29(4):64–71, 2000. ISSN 0163-5808. doi: 10.1145/369275.369291. URL https://doi.org/10.1145/369275.369291.

Vijay Janapa Reddi, Christine Cheng, David Kanter, Peter Mattson, Guenther Schmuelling, Carole-Jean Wu, et al. MLPerf Inference Benchmark. *arXiv preprint arXiv:1911.02549*, 2019.